

UE25CS251A
DIGITAL LOGIC AND COMPUTER DESIGN

TEAM DLCD

Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

Memory Arrays

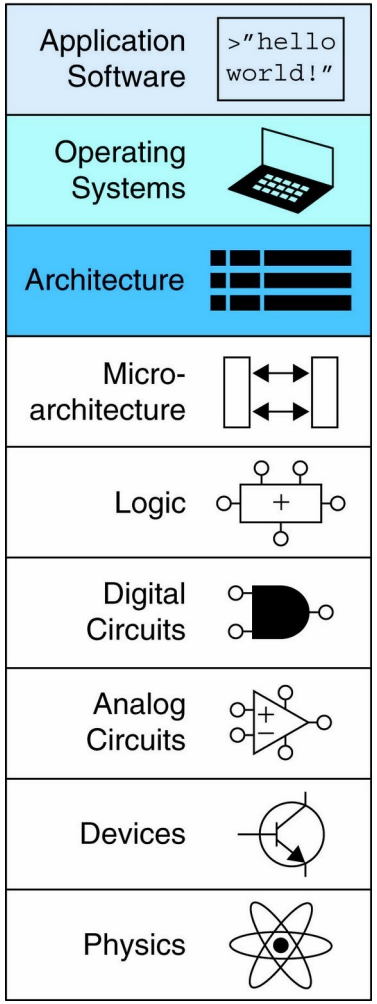
TEAM DLCD
Department of CSE and CSE (AI & ML)

DIGITAL LOGIC & COMPUTER DESIGN

TOPICS

5.5 Memory Arrays

- 5.5.1 Overview
- 5.5.2 Dynamic Random Access Memory (DRAM)
- 5.5.3 Static Random Access Memory (SRAM)
- 5.5.4 Area and Delay
- 5.5.5 Register Files
- 5.5.6 Read Only Memory
- 5.5.7 Logic Using Memory Arrays
- 5.5.8 Memory HDL



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

5.5 Memory Arrays — Overview

A memory array stores many words using a 2-D array of 1-bit cells.

- N address bits select one of 2^N words.
- Each word contains M data bits.
- Size = depth × width = 2^N × M bits.
- Memories are specified by size and by the number/type of ports.

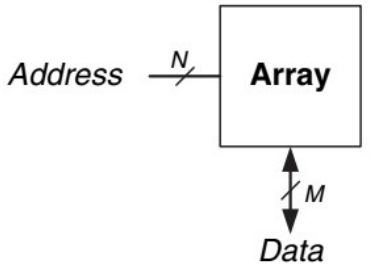


Figure 5.38 Generic memory array symbol

Example: 10 address bits + 32-bit data
→ 2^10 = 1024 words
→ 1024 × 32 = 32 Kb

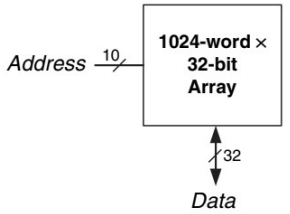


Figure 5.40 32 Kb array: depth = 2^{10} = 1024 words, width = 32 bits

Each bit cell connects to a wordline and a bitline. A decoder activates one wordline for the selected address.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

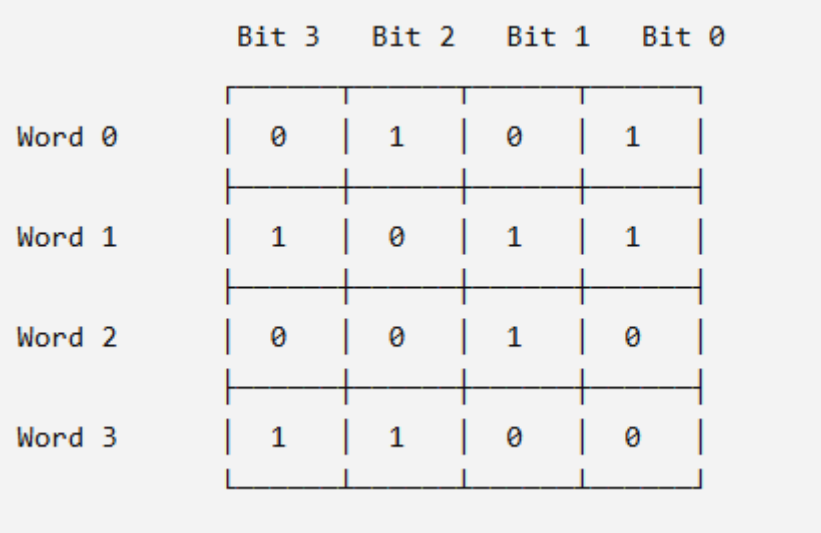
DIGITAL LOGIC & COMPUTER DESIGN

5.5 Memory Arrays — Overview

What is a memory array?
A memory is made up of many tiny **1-bit storage cells** arranged in rows and columns.
Think of it like a spreadsheet:

- **Each row = one word**
- **Each column = one bit**
- A collection of bits in one row forms a **word**

For example, if each word is 4 bits:



What does N address bits mean?
Suppose we have **N address bits**.
Each address bit can be either 0 or 1.
Therefore, the number of different addresses is:

$$2^N$$

So, **N address bits can select 2^N different words.**
For example:

- 2 address bits $\rightarrow 2^2 = 4$words
- 3 address bits $\rightarrow 2^3 = 8$words
- 10 address bits $\rightarrow 2^{10} = 1024$words

So:
10 address bits mean the memory has 1024 words.

What does M data bits mean?
If each word contains **M bits**, then each selected row has M bits.
For example:
32-bit data means:
One word = 32 bits
Therefore, if we have 1024 words:

$$1024 \times 32 = 32768 \text{ bits}$$

Since:

$$32768 = 32 \times 1024 = 32K$$

the memory size is:

$$\boxed{32 \text{ Kbits}}$$

Important: This is **32 Kb (kilobits)**, not 32 KB (kilobytes).

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

5.5 Memory Arrays — Overview

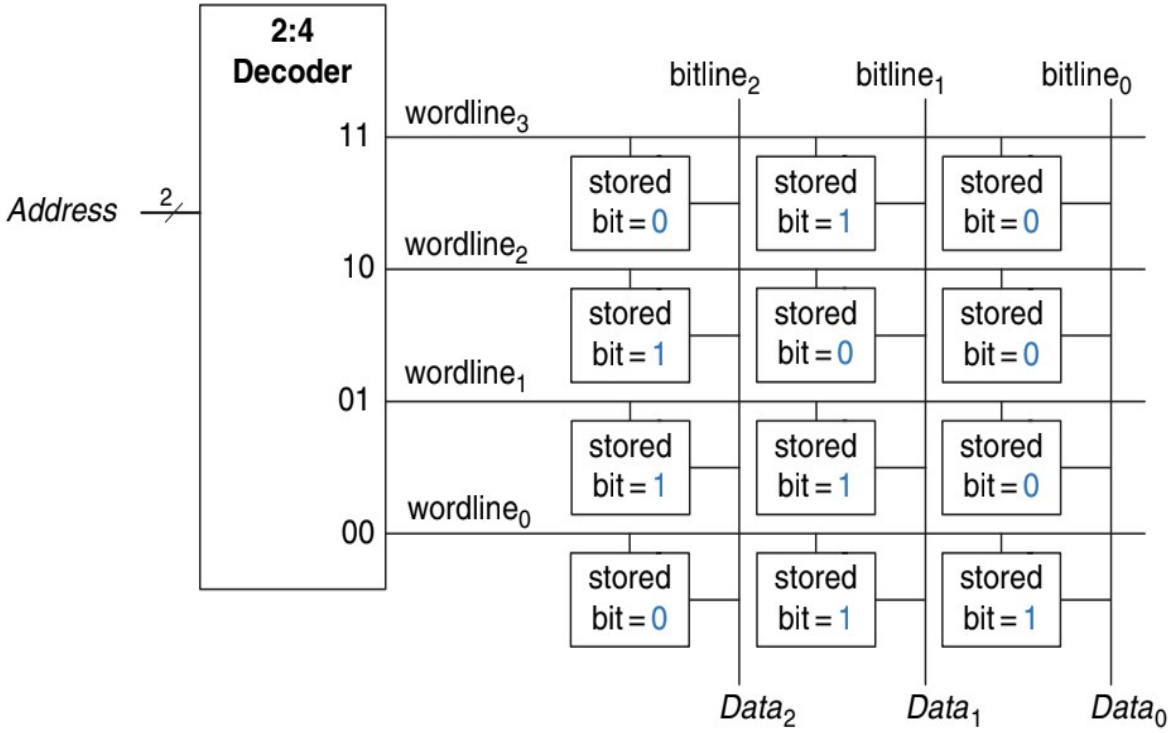


Figure 5.42 4 × 3 memory array

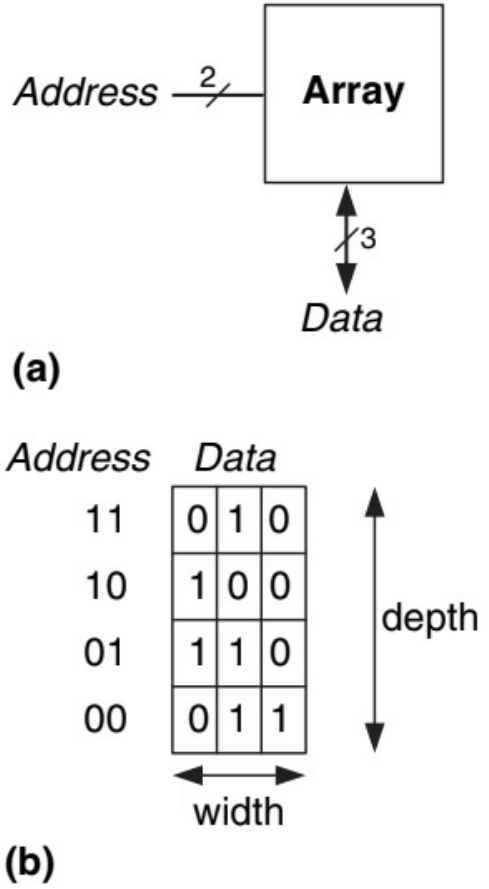


Figure 5.39 4 × 3 memory array: (a) symbol, (b) function

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Memory Array: Bit Cell Organization

Memory Bit Cell

- A memory array is built as an array of **bit cells**, where each bit cell stores **1 bit**.
- Each bit cell is connected to:
  - **Wordline** – selects the bit cell.
  - **Bitline** – transfers data to or from the bit cell.
- For each combination of address bits, **one wordline is activated**.
- When the wordline is **HIGH**, the selected bit cells are connected to their bitlines.
- When the wordline is **LOW**, the bitline is disconnected from the bit cell.
- The circuitry used to store the bit depends on the **type of memory**.

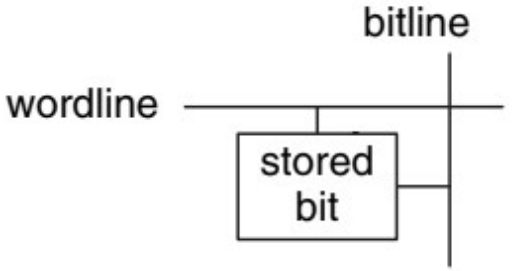


Figure 5.41 Bit cell

Wordline = Selects the row

Bitline = Carries the data

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Reading and Writing a Bit Cell

Read Operation

1. Bitline is initially left **floating (Z)**.
2. The **wordline is turned ON**.
3. The stored bit drives the bitline to **0 or 1**.

Bit Cell→Bitline

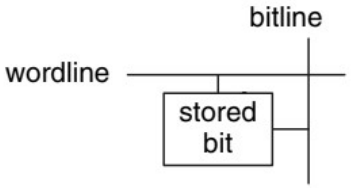


Figure 5.41 **Bit cell**

Write Operation

1. Bitline is **strongly driven** to the desired value.
2. The **wordline is turned ON**.
3. The strongly driven bitline **overpowers the existing bit**.
4. The desired value is stored in the bit cell.

Bitline→Bit Cell

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory Array Organization

Internal Organization of a 4 × 3 Memory Array

- A **4 × 3 memory array** contains:
  - **4 words (rows)**
  - **3 bits per word**
- Practical memories are much larger, but their operation can be understood using this smaller array.
- A **decoder** selects one wordline based on the given address.
- The selected row of bit cells is connected to the **Data bitlines**.

Read Operation

- Wordline is asserted.
- The selected row of bit cells drives the bitlines **HIGH or LOW**.
- The resulting values appear on the **Data bitlines**.

Write Operation

- Data bitlines are first driven to the required values.
- The appropriate wordline is then asserted.
- The bitline values are stored in the selected row.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory Array Organization

Reading Address 10

1. Address = **10**
2. Decoder asserts **$wordline_2$**.
3. Bitlines are initially left **floating**.
4. The selected row contains **100**.
5. Therefore, **100** appears on the Data bitlines.

10→Wordline2→100

Writing 001 to Address 11

1. Address = **11**
2. Data bitlines are driven to **001**.
3. Decoder asserts **$wordline_3$**.
4. The value **001** is stored in that row.

11+001→Wordline3→001 stored

READ: Wordline ON→Stored data drives bitlines

WRITE: Drive bitlines→Wordline ON→Store data

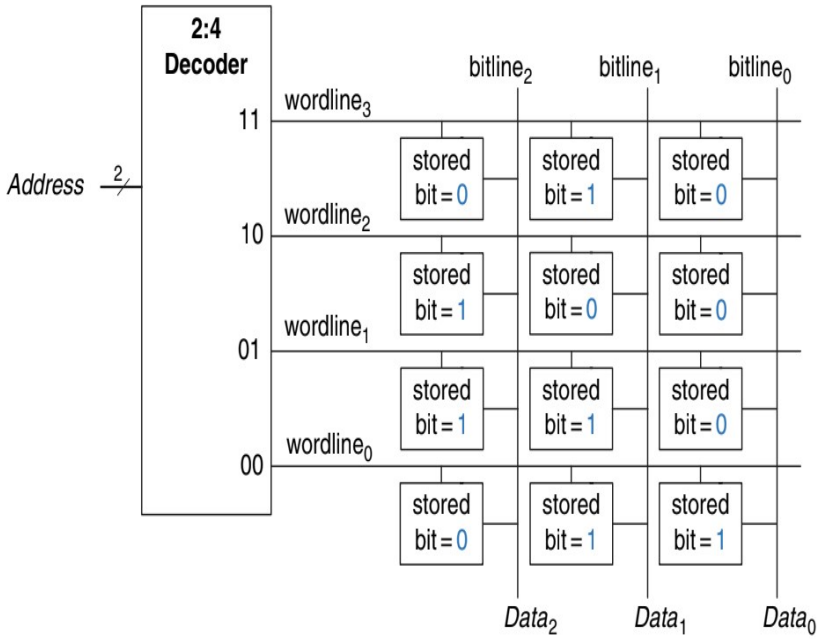


Figure 5.42 4 × 3 memory array

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Memory Array: Depth, Width(Word size)

Depth, width and bit cells

- Depth = number of words (rows).
- Width = number of bits per word (columns).

Example: 4 × 3 memory
- 2 address bits select 1 of 4 words.
- Each word contains 3 bits.
- Total storage = 4 × 3 = 12 bits.

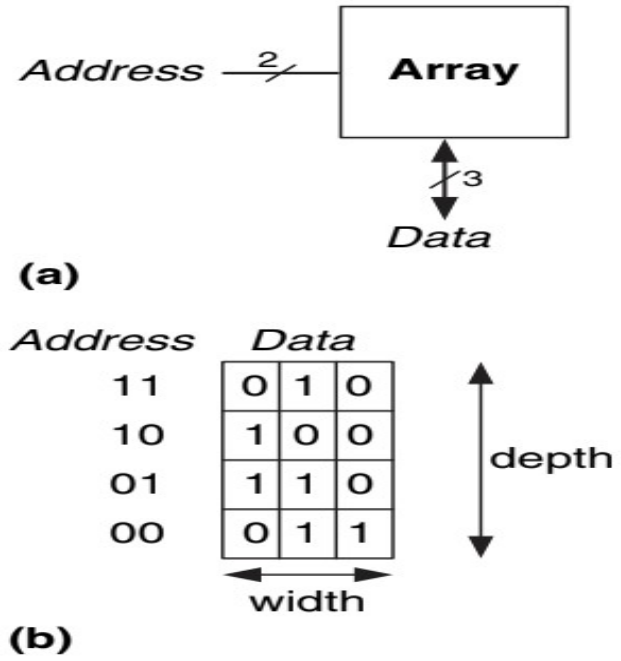


Figure 5.39 4 × 3 memory array: (a) symbol, (b) function

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Memory Ports & Read/Write Example

A memory port provides access to one memory address. Memories may be single-ported or multiported.

Example: three-port memory

- Two read ports can read addresses A1 and A2 simultaneously.
- One write port writes WD3 to address A3 when WD3 is asserted on the rising clock edge.

Practical example — register file:

A processor may read two source registers while writing one destination register in the same cycle.

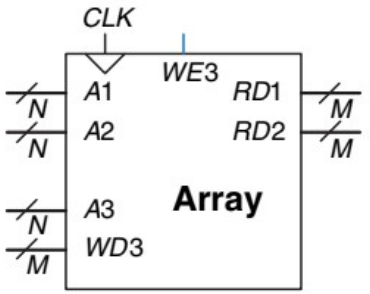


Figure 5.43 Three-ported memory

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Memory Array Classification

Memory arrays are specified by:

- **Size:** Depth × Width
- **Number of ports**
- **Type of ports**

All memory arrays store data using an **array of bit cells**, but they differ in **how the bits are stored**.

RAM vs ROM: The broadest classification of memory is:

RAM	ROM
Random Access Memory	Read Only Memory
Volatile	**Nonvolatile**
Loses data when power is turned OFF	Retains data without power
Historically associated with read/write memory	Historically associated with read-only memory

Both RAM and ROM can be **randomly accessed**. Modern ROMs can also be written in various ways.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DRAM vs SRAM

There are two major types of RAM:

1. **DRAM (Dynamic RAM)**

- Stores each bit as charge on a capacitor.
- Charge leaks away, so periodic refresh is required.
- Reading disturbs/destroys the capacitor value, so the data is restored after a read.
- The simplified cell uses about 1 transistor per bit → very dense.

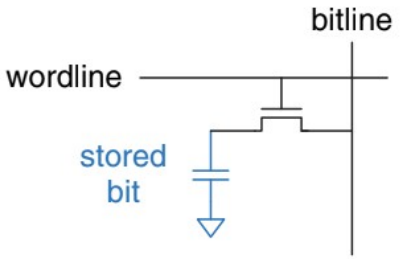


Figure 5.44 DRAM bit cell

1. **SRAM (Static RAM)**

- Stores the bit using cross-coupled inverters.
- No periodic refresh is required while power is applied.
- The cross-coupled inverters restore the value if noise disturbs it.
- A typical SRAM bit cell uses 6 transistors → larger than DRAM but faster.

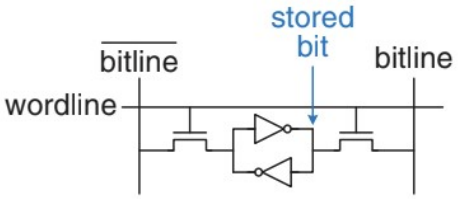


Figure 5.46 SRAM bit cell

Book comparison: flip-flop ≈ 20 transistors/bit, SRAM ≈ 6, DRAM ≈ 1.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

DRAM vs SRAM

1. Why is DRAM called "Dynamic"?

A DRAM cell stores a bit as **electrical charge on a capacitor**.

Charged capacitor → **1**

Discharged capacitor → **0**

But the capacitor gradually **leaks its charge**.

So:

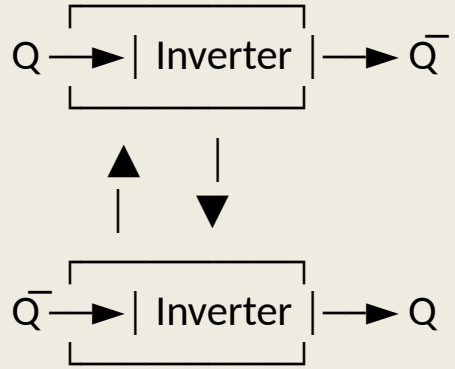
Stored charge → leaks with time → data may be lost → refresh is required

That's why it is **Dynamic**: the stored value needs to be periodically refreshed.

2. Why is SRAM called "Static"?

SRAM uses **cross-coupled inverters**.

The two inverters continuously reinforce each other's state:



If Q = 1, the feedback keeps Q = 1 and $\overline{Q}$ = 0.

If Q = 0, the feedback keeps Q = 0 and $\overline{Q}$ = 1.

Therefore, **as long as power is supplied, SRAM does not need periodic refresh.**

3. Why is SRAM faster than DRAM?

The important difference is:

DRAM → charge must be sensed from a capacitor

SRAM → logic state is already maintained by the transistor circuit

So SRAM can generally be accessed faster, but its cell requires more transistors and therefore occupies more chip area.

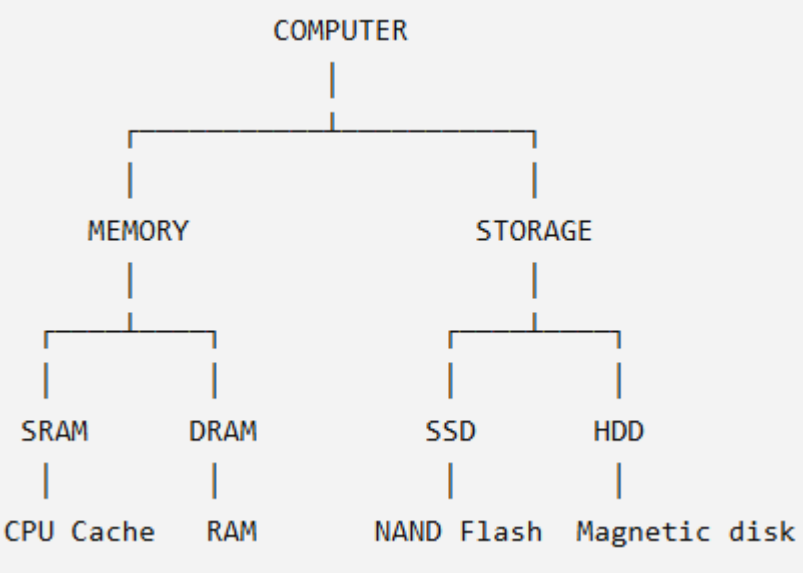
© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

DRAM vs SRAM

DRAM vs SRAM vs Flip-Flop

Feature	DRAM	SRAM	Flip-Flop
Basic storage	**Capacitor charge**	**Cross-coupled inverters**	Feedback logic
Typical transistors/bit	**1T + capacitor**	**6T**	**~20T**
Refresh required?	**Yes**	**No**	**No**
Read operation	**Destructive** → restore needed	Non-destructive	Non-destructive
Speed	Slower	Faster	Very fast
Density	**Highest**	Lower	Lowest
Main use	Main memory	Cache	Registers/sequential logic



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Volatile Memory - Area & Delay

Flip-flops, SRAM, and DRAM are all volatile memories, but they differ in **area, latency, and throughput**.

Memory	Transistors/Bit	Relative Latency	Key characteristic
Flip-flop	~20	Fast	Output available immediately
SRAM	6	Medium	Faster access than DRAM
DRAM	1	Slow	High density

Why?

- More transistors generally mean **more area, power, and cost**.
- **Flip-flops:** data is immediately available at the output, but require at least ~20 transistors.
- **SRAM:** uses 6 transistors per bit.
- **DRAM:** uses only 1 transistor per bit, making it more area-efficient.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Why is DRAM Slower?

DRAM stores a bit as charge on a capacitor.

As shown in Figure 5.45(a), when the capacitor is charged to VDD, the stored bit is 1; when it is discharged to GND (Figure 5.45(b)), the stored bit is 0.

During a read:

1. Charge moves from the **capacitor → bitline**.
2. The bitline is **not actively driven by a transistor**.
3. Charge movement is relatively slow.
4. Therefore, **DRAM has higher latency than SRAM**.

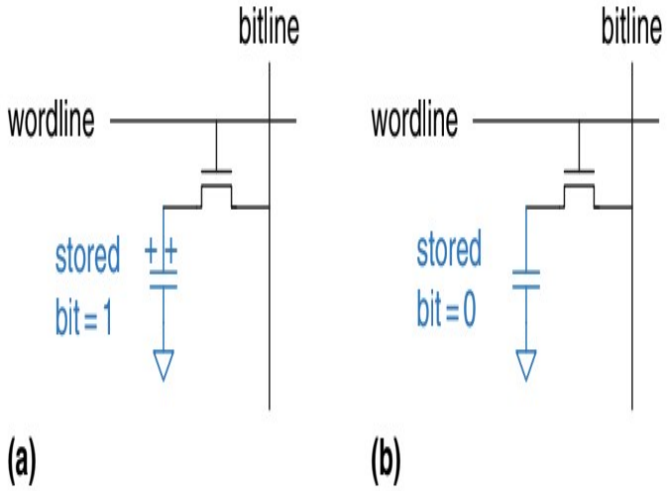


Figure 5.45 DRAM stored values

DRAM also has lower throughput

DRAM must:

- **Periodically refresh** its stored data.
- Refresh the data **after a read**.

Therefore, its throughput is fundamentally lower than SRAM.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

SDRAM & DDR SDRAM

Technologies were developed to improve DRAM throughput:

SDRAM - Synchronous DRAM

- Uses a **clock**
- Pipelines memory accesses

DDR SDRAM - Double Data Rate SDRAM

- Transfers/accesses data on **both rising and falling edges** of the clock.
- Therefore, it can **double the throughput for a given clock speed**.

It should be noted that DDR was first standardized in **2000**, initially operating at **100–200 MHz**. Later standards such as **DDR2, DDR3, and DDR4** increased clock speeds.

Important take away DRAM → lower area per bit, but higher latency and lower throughput.
SRAM → faster, but uses more transistors.
Flip-flop → fastest, but largest per-bit implementation.

Also remember: **Larger memories tend to be slower than smaller memories**, assuming other factors are the same. The appropriate memory type depends on the design's **speed, cost, and power constraints**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Register Files

Register file = a group of registers used to store **temporary variables** in a digital system.

- Usually implemented as a **small, multi-ported SRAM array**
- More compact than building the same registers using an array of flip-flops
- A register file can allow **multiple registers to be accessed simultaneously**.

Example: 32 × 32 Three-Ported Register File

The textbook's Figure 5.47 shows a **32-register × 32-bit** register file with:

- **2 read ports:**
  - A1 → RD1
  - A2 → RD2
- **1 write port:**
  - A3 → WD3
- CLK controls the write operation.
- Each address is **5 bits**, because: 2^5 = 32

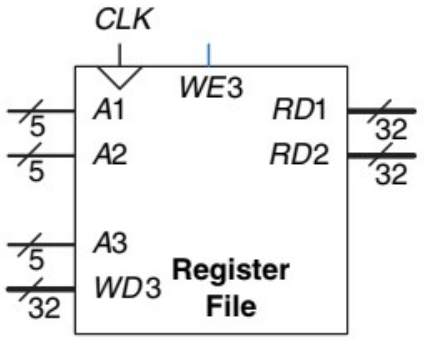


Figure 5.47 32 × 32 register file with two read ports and one write port

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Register Files

Therefore:

Two registers can be read and one register can be written simultaneously.

Numerical

For a **32 × 32 register file**:

Number of registers=32

Bits per register=32

Address bits=log2(32)=5

Total storage:

32×32=1024 bits

Register File → Small multi-ported SRAM → Temporary storage → Multiple simultaneous accesses

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Read-Only Memory (ROM)

ROM stores each bit based on the presence or absence of a transistor.

How a ROM bit cell works

1. The **bitline is weakly pulled HIGH**.
2. The **wordline is turned ON**.
3. If a transistor is **present** → it pulls the bitline **LOW** → stored bit = **0**.
4. If a transistor is **absent** → bitline remains **HIGH** → stored bit = **1**.

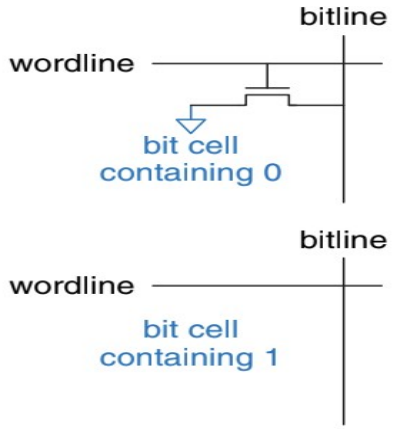


Figure 5.48 ROM bit cells containing 0 and 1

Key point

ROM is **non volatile** because its bit cell is a **combinational circuit** and has no state that is lost when power is removed.

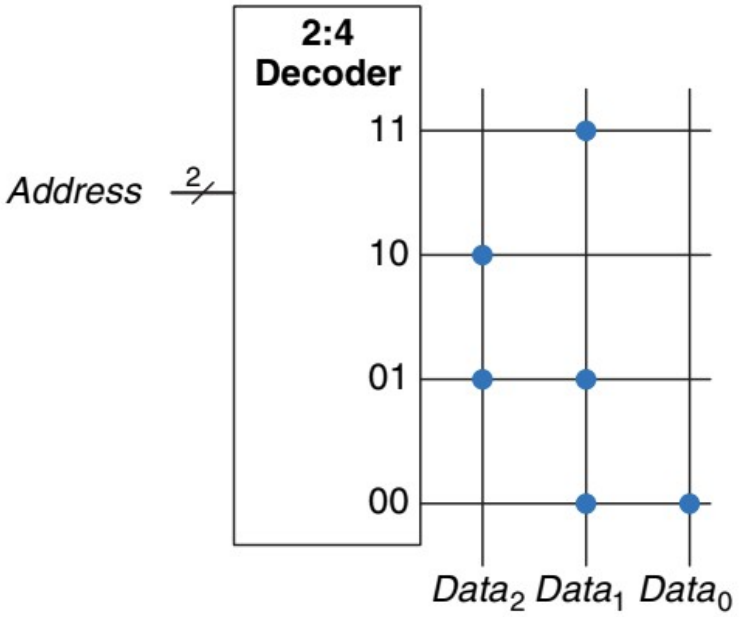
© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

ROM Dot Notation

ROM contents can be represented using **dot notation**.

- **Rows → Wordlines / Addresses**
- **Columns → Bitlines / Data bits**
- **Dot at an intersection → stored bit = 1**
- **No dot → stored bit = 0**



Example: 4 × 3 ROM

A **4-word × 3-bit ROM** has: 2^2=4 words

and each word contains:3 bits

So the total storage is:

4×3=12 bits

For the example shown in Figure 5.49, the top wordline has a dot only on **Data1**, so the corresponding stored word is **010**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

ROM

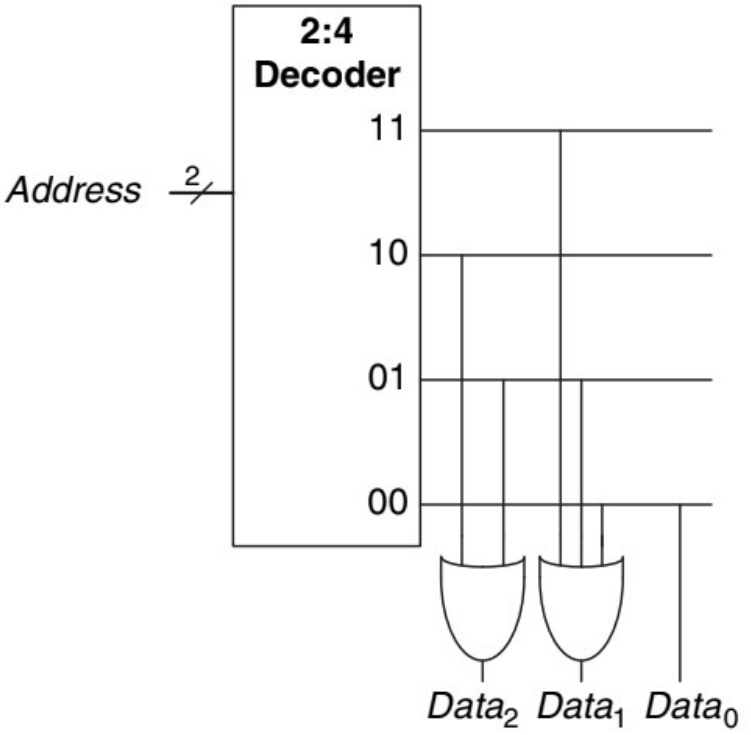


Figure 5.50 4 × 3 ROM implementation using gates

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

ROM as Logic

A ROM can also be viewed as a **two-level logic circuit**:

AND gates → OR gates

First level: AND gates

- Generate all possible **minterms**
- Therefore, they function as a **decoder**

Second level: OR gates

- Combine selected minterms
- Produce the required output functions

So conceptually: **Address → Decoder → OR gates → Data**

Important point: A ROM can implement **any two-level logic function**.

In practice, ROMs use **transistors instead of logic gates** to reduce size and cost.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Types of ROM

ROMs differ mainly in **how their contents are programmed and erased**.

1. Mask ROM

- Contents are specified **during manufacturing**
- A transistor is either present or absent in each bit cell.
- Cannot be changed by the user.

2. PROM - Programmable ROM

- Transistors are initially present.
- User can program the memory.
- In the fuse-programmable version, programming selectively **blows fuses**.
- Once programmed, it cannot be reversed.

PROM = One-Time Programmable ROM (OTP)

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Fuse-Programmable ROM

Programming operation

Fuse intact:

- Transistor connected to GND
- Cell stores **0**

Fuse blown:

- Transistor disconnected from GND
- Cell stores **1**

A **high voltage** is applied to selectively blow the required fuses.

Because a blown fuse cannot be repaired:

PROM is one-time programmable

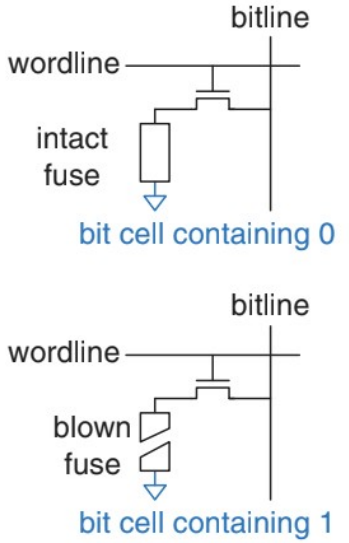


Figure 5.51 **Fuse-programmable ROM bit cell**

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Reprogrammable ROM

EPROM — Erasable Programmable ROM

Uses a **floating-gate transistor**.

Programming:

- High voltage causes electrons to move onto the floating gate.
- This changes the transistor's state.

Erasing:

- Exposed to intense **UV light**.
- UV light removes the electrons from the floating gate.

So:

EPROM → Program electrically → Erase using UV light

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

EEPROM & Flash Memory



EEPROM

Electrically Erasable Programmable ROM

- Can be programmed electrically.
- Can also be erased electrically.
- **Individual bits/cells can be erased.**
- No UV light required.

Flash Memory

- Uses similar principles to EEPROM.
- Erases **larger blocks of bits** rather than individual cells.
- Requires fewer erasing circuits.
- Therefore, it is **cheaper** than EEPROM.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Comparison

Type	Programmable?	Erasable?	Erasing method
Mask ROM	During manufacturing	No	—
PROM	Once	No	—
EPROM	Yes	Yes	UV light
EEPROM	Yes	Yes	Electrical
Flash	Yes	Yes	Electrical, block-wise

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Memory Arrays as Logic

Memory arrays are primarily used for **data storage**, but they can also implement **combinational logic functions**.

For a memory of: 2^N-words×M-bits

it can implement **any combinational function with:**

- **N inputs**
- **M outputs**

Why?

- The **N inputs** are used as the **address**.
- Each possible input combination corresponds to one memory row.
- The **M data bits** stored in that row represent the outputs.

So:

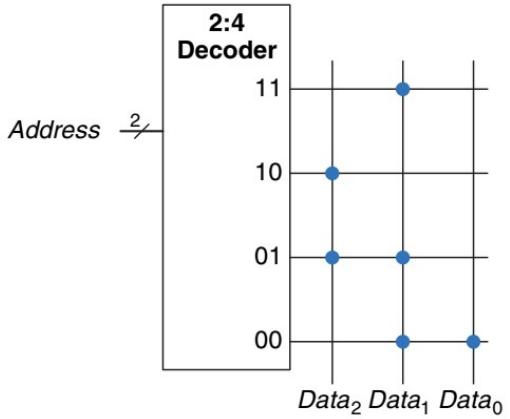
Inputs → Address → Memory Array → Data Outputs

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Example — ROM as Combinational Logic

The **4 × 3 ROM** from Figure 5.49 has:

- 4 words → 2^2, so **2 inputs**
- 3-bit data → **3 outputs**



Therefore, it implements: **3 functions of 2 inputs**

For this particular ROM:

- **Data2 = XOR** of the two address inputs
- **Data0 = NOR** of the two address inputs
- Data1 is another combinational function determined by the stored ROM contents.

Key idea

The ROM contents essentially represent the truth table of the desired logic functions.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Lookup Tables (LUTs)

A memory array used to perform logic is called a:

Lookup Table (LUT)

Instead of constructing a function using individual logic gates, we **store the output values in memory**.

How a LUT works

1. Inputs are applied as an **address**.
2. The address selects a particular **row**.
3. The stored data in that row gives the **output**.

Input combination → Address → Selected row → Output

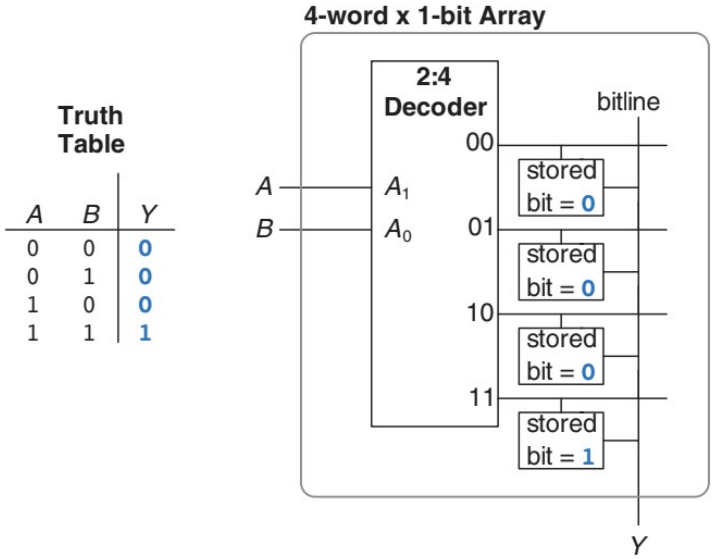


Figure 5.52 4-word × 1-bit memory array used as a lookup table

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Example — LUT for Y=AB

Lets uses a **4-word × 1-bit memory** to implement: Y=AB

Since there are **2 inputs**:2^2=4 possible input combinations

A	B	Address	Y = AB
0	0	00	0
0	1	01	0
1	0	10	0
1	1	11	1

The LUT simply stores: 0, 0, 0, 1

For **N inputs** and **M outputs**: **LUT size=2N×M**

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q1. An N-bit binary counter has how many possible states?

A. N
B. 2N
C. N^2
D. 2^N

Q2. What happens to a binary counter on each rising clock edge after reset is released?

A. Q remains constant
B. Q decreases by 1
C. Q increases by 1
D. Q becomes all 1s

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q3. A shift register with N flip-flops can act as which converter?
A. Serial-to-parallel
B. Analog-to-digital only
C. Frequency-to-voltage
D. Decimal-to-BCD only

Q4. In a shift register with parallel load, what happens when Load = 1?
A. The register is cleared
B. D inputs are loaded into the flip-flops
C. The clock is disabled permanently
D. The register shifts without loading D

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q5. A memory has 10 address bits and 8 data bits. What is its organization?
A. 10 × 8 bits
B. 100 × 8 bits
C. 1024 × 8 bits
D. 2048 × 10 bits

Q6. During a memory read, what does the decoder primarily do?
A. Adds two data words
B. Activates the wordline for the selected address
C. Refreshes every bit cell
D. Converts binary to decimal

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q7. Which statement about DRAM is correct?
A. It stores data in cross-coupled inverters and never needs refresh
B. It stores charge on a capacitor and requires refresh
C. It is nonvolatile
D. It uses more transistors per bit than a flip-flop

Q8. Which statement about SRAM is correct?
A. It requires periodic refresh like DRAM
B. It stores the bit using cross-coupled inverters
C. A read always destroys the stored bit
D. It is nonvolatile

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Q9. A typical processor register file in the textbook has which port arrangement?

A. One read port and no write port
B. Two read ports and one write port
C. Three write ports and no read ports
D. One read/write port only

Q10. A 2^N-word × M-bit memory used as a LUT has:

A. N input/address bits and M output bits
B. M input bits and N output bits
C. N output bits only
D. One fixed output bit

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Answer Key:

Q1: D Q2: C Q3: A Q4: B Q5: C Q6: B
Q7: B Q8: B Q9: B Q10: A

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.



THANK YOU

Team DLCD

Department of CSE and CSE(AI & ML)